DevSecOps and Cloud Computing · FIAP · Engenharia de Software

CP05 — O pipeline DevSecOps completo

Pipeline de referência com Gitleaks, Semgrep e Trivy, e as provas de sabotagem de cada barreira

Item	Valor
Grupo	Breno Silva — RM97864 Enrico Marquez — RM99325 Gustavo Dias — RM550820 Joel Barros — RM550378 Leonardo Moreira — RM550988
Usuário GitHub (repositório)	jonzaka
Repositório	https://github.com/jonzaka/cp05-devsecops-pipeline
Workflow	`.github/workflows/security.yml`
Data	23/10/2026

1. O cenário e a solução

Uma chave de API vazou num commit público da Arcanjosa e a diretoria decretou pipeline de segurança obrigatório em todos os repositórios. O pipeline de referência construído aqui roda em todo `push` e em todo `pull_request` e tem três barreiras independentes, uma por job do GitHub Actions. Se qualquer uma falhar, o workflow fica vermelho e o pull request não pode ser mesclado.

Job	Ferramenta	O que barra	Critério de falha
`gitleaks`	Gitleaks	Segredos no histórico do git: chaves AWS, tokens, senhas, chaves genéricas de alta entropia	Qualquer segredo em qualquer commit do push ou do PR
`semgrep`	Semgrep	Código vulnerável (SAST): packs `p/javascript` e `p/security-audit` mais a regra `eval-detected`	Qualquer achado (`--error`)
`trivy`	Trivy	Vulnerabilidades conhecidas (CVE) na imagem Docker construída a partir do `Dockerfile`	Qualquer CVE de severidade CRITICAL com correção disponível

O repositório contém o portal de status da Arcanjosa (site estático em HTML, CSS e JavaScript puro), o `Dockerfile` que o serve com Nginx e o workflow abaixo.

security.yml

```
name: Security Pipeline

# Roda em todo push e em todo pull request: nenhum código chega à main
# sem passar pelas três barreiras.
on: [push, pull_request]

permissions:
  contents: read

jobs:
  # Barreira 1 — segredos (chaves, tokens, senhas) no histórico do git.
  gitleaks:
    name: Gitleaks (segredos)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          # Histórico completo: o Gitleaks varre todos os commits do push/PR,
          # não só o estado final dos arquivos.
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

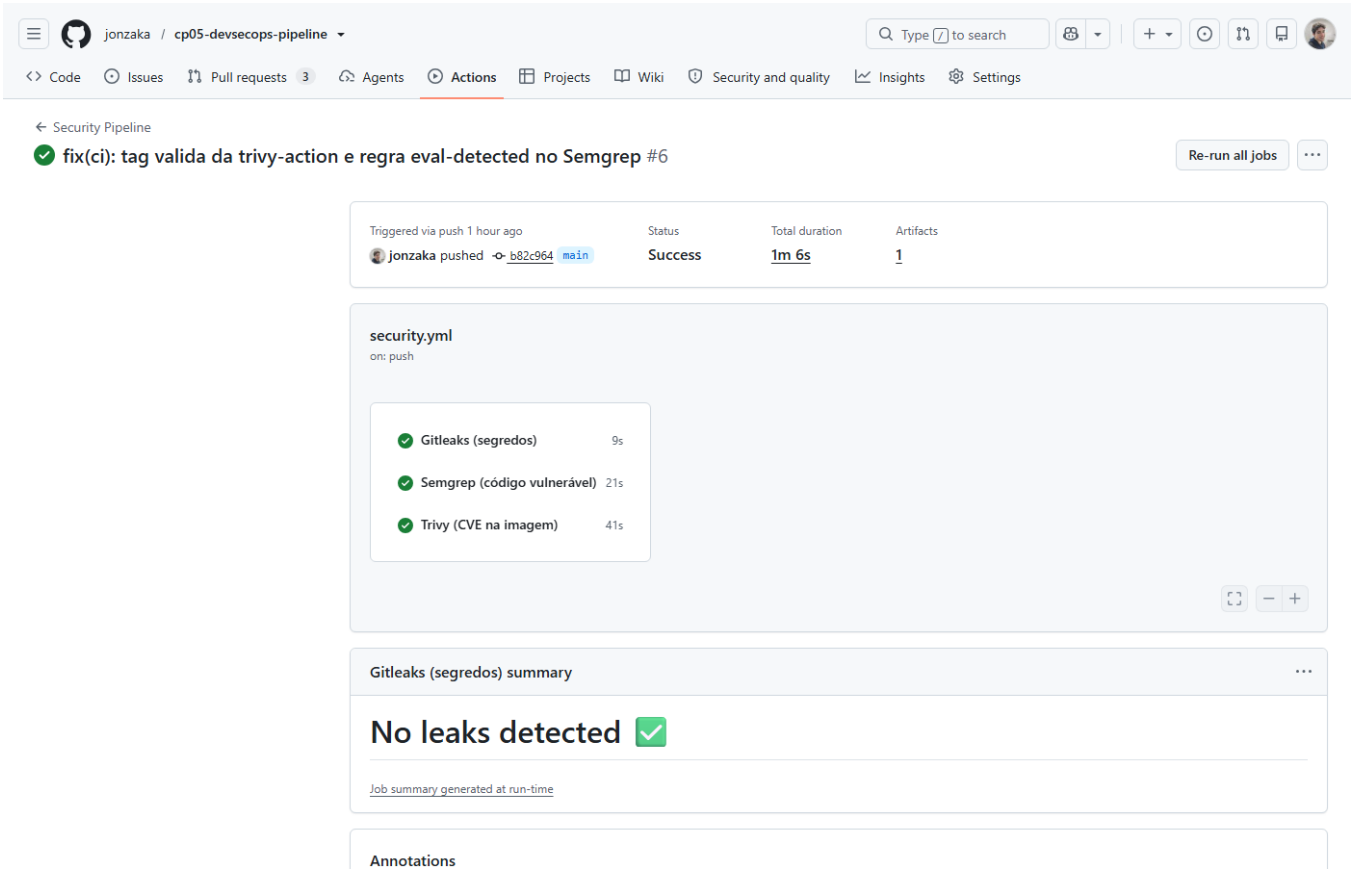
  # Barreira 2 — código vulnerável (SAST).
  semgrep:
    name: Semgrep (código vulnerável)
    runs-on: ubuntu-latest
    container:
      image: semgrep/semgrep
    steps:
      - uses: actions/checkout@v4
      - name: Analisar código
        run: >
          semgrep scan
          --config p/javascript
          --config p/security-audit
          --config r/javascript.browser.security.eval-detected
          --error
          --metrics=off

  # Barreira 3 — vulnerabilidades (CVE) na imagem Docker.
  trivy:
    name: Trivy (CVE na imagem)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Construir imagem
```

```
        run: docker build -t arcanjosa-portal:${{ github.sha }} .
      - name: Escanear imagem
        uses: aquasecurity/trivy-action@v0.36.0
        with:
          image-ref: arcanjosa-portal:${{ github.sha }}
          format: table
          severity: CRITICAL
          exit-code: "1"
          ignore-unfixed: true
```

2. Linha de base: a main passa nas três barreiras

Antes de sabotar, o pipeline precisa ficar verde no código limpo. Caso contrário uma falha na prova poderia ser defeito do pipeline e não detecção de verdade. O run abaixo é o da `main`: os três jobs passaram.



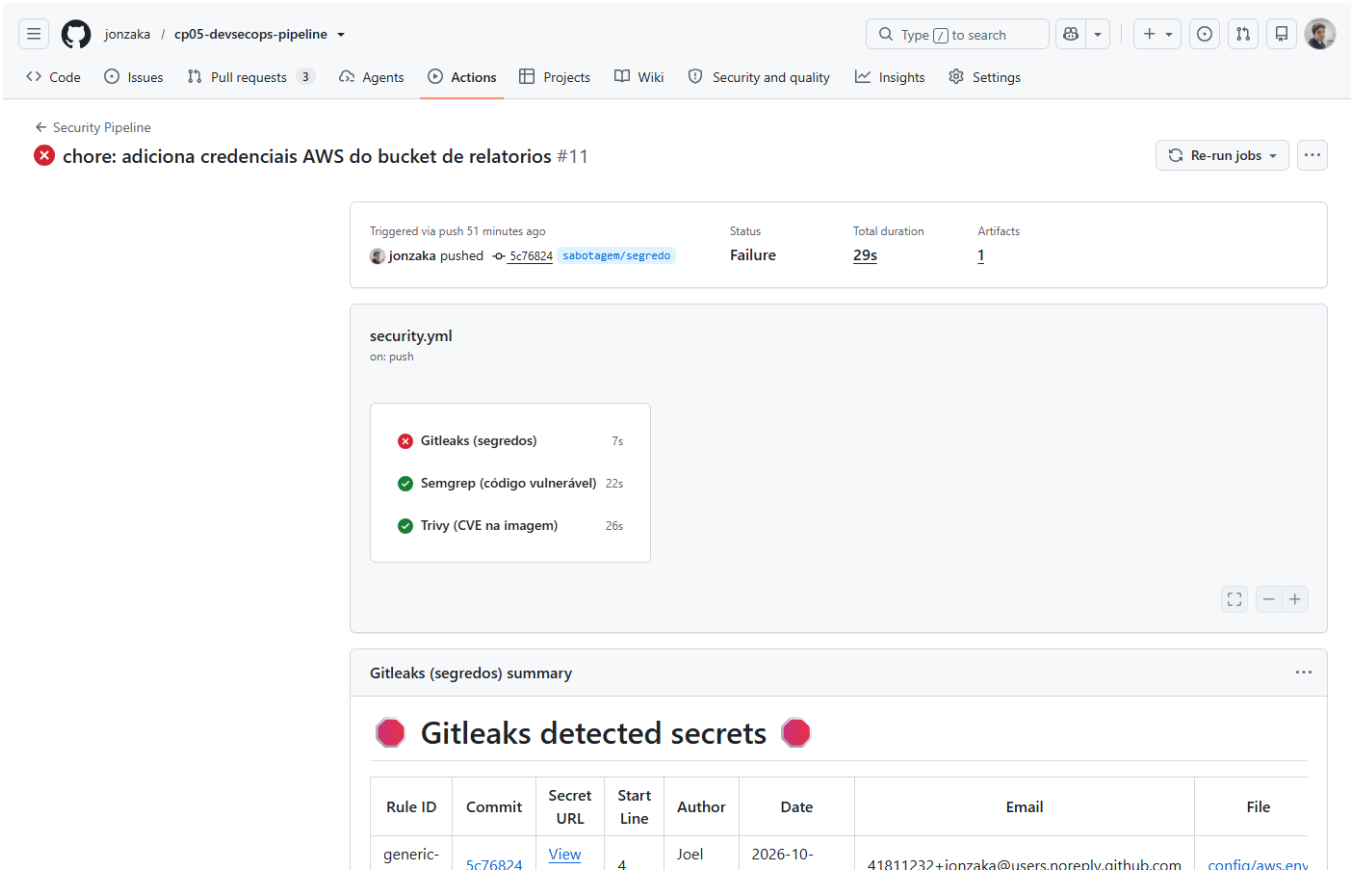
Run #6 da main: Gitleaks, Semgrep e Trivy verdes. Commit b82c964, usuário jonzaka.

> **Decisão registrada durante a construção.** A imagem base começou como `nginx:1.27-alpine`. O Trivy a reprovou por um CVE crítico do OpenSSL (CVE-2026-31789, libcrypto3 3.3.3 com correção em 3.3.7). A base foi trocada por `nginx:1.29-alpine`, sem CVE crítico com correção. Ou seja: a barreira 3 já tinha agido antes mesmo da sabotagem planejada.

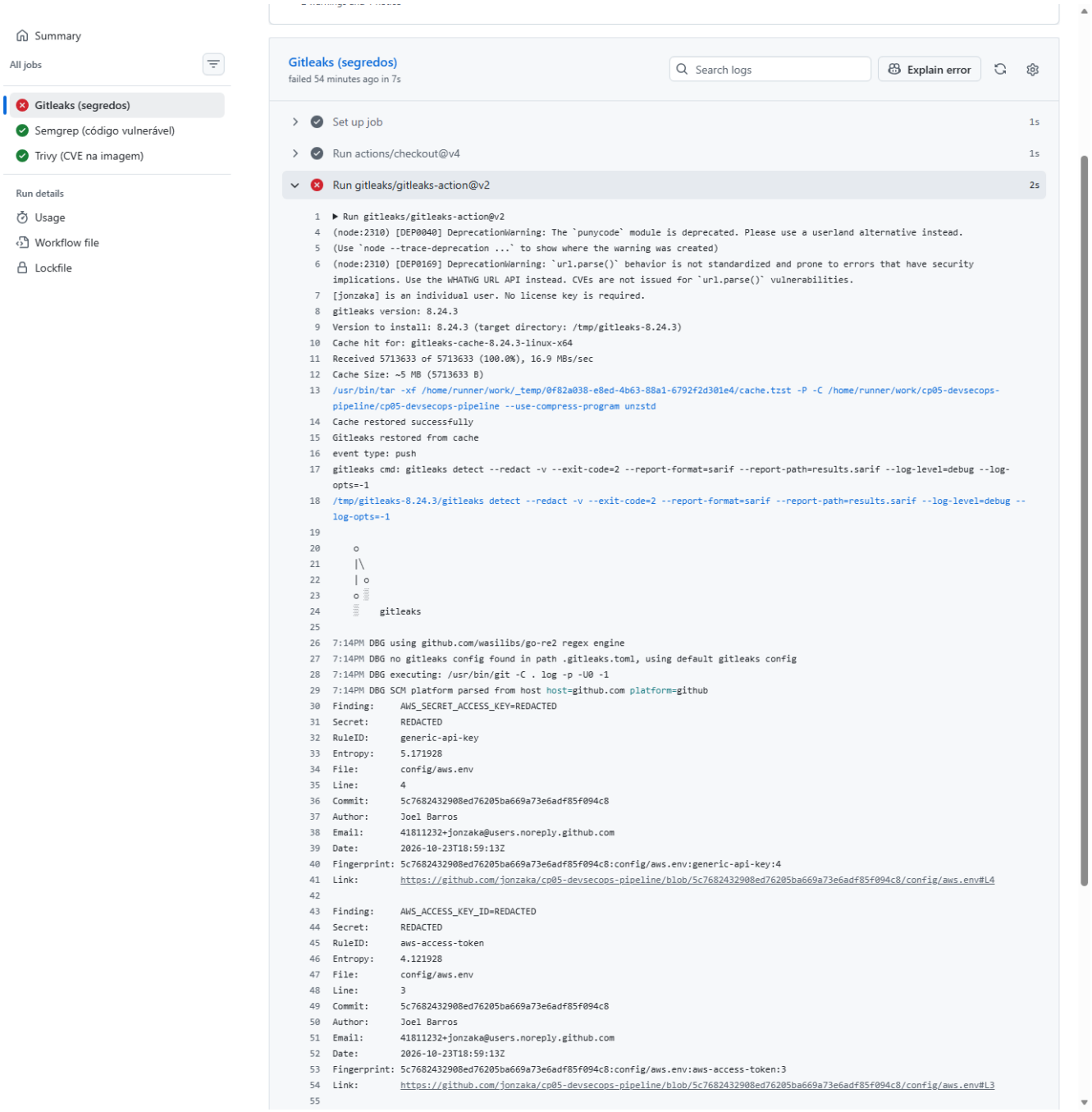
3. Prova 1 — segredo falso barrado pelo Gitleaks

Sabotagem. No branch `sabotagem/segredo` foi feito um commit com o arquivo `config/aws.env` contendo um par de credenciais AWS falsas (um `AWS_ACCESS_KEY_ID` no formato `AKIA...` e um `AWS_SECRET_ACCESS_KEY` de 40 caracteres). As chaves não existem na AWS; têm só o formato real.

Resultado. O job `gitleaks` falhou em 7 segundos e os outros dois passaram. O Gitleaks varreu o histórico completo (`fetch-depth: 0`) e encontrou dois vazamentos no mesmo commit: a chave de acesso pela regra `aws-access-token`, que reconhece o prefixo `AKIA` da AWS, e a chave secreta pela regra `generic-api-key`, que não conhece o formato mas mede a entropia do valor (5,17 bits por caractere, típico de material criptográfico, não de texto).



Run #11 do branch sabotagem/segredo: só o Gitleaks reprovou.



Job Gitleaks: o passo "Run gitleaks/gitleaks-action@v2" falhou.

Trecho do log do job

```
Finding:     AWS_SECRET_ACCESS_KEY=REDACTED
Secret:      REDACTED
RuleID:      generic-api-key
Entropy:     5.171928
File:        config/aws.env
Line:        4
Commit:      5c7682432908ed76205ba669a73e6adf85f094c8
Author:      Joel Barros
Email:       41811232+jonzaka@users.noreply.github.com
Date:        2026-10-23T18:59:13Z
Fingerprint: 5c7682432908ed76205ba669a73e6adf85f094c8:config/aws.env:generic-api-key:4
Link:        https://github.com/jonzaka/cp05-devsecops-pipeline/blob/5c7682432908ed76205ba669a73e6adf85f094c8/config/a
  ws.env#L4
Finding:     AWS_ACCESS_KEY_ID=REDACTED
Secret:      REDACTED
RuleID:      aws-access-token
Entropy:     4.121928
File:        config/aws.env
Line:        3
Commit:      5c7682432908ed76205ba669a73e6adf85f094c8
Author:      Joel Barros
Email:       41811232+jonzaka@users.noreply.github.com
Date:        2026-10-23T18:59:13Z
Fingerprint: 5c7682432908ed76205ba669a73e6adf85f094c8:config/aws.env:aws-access-token:3
Link:        https://github.com/jonzaka/cp05-devsecops-pipeline/blob/5c7682432908ed76205ba669a73e6adf85f094c8/config/a
  ws.env#L3
7:14PM INF 1 commits scanned.
7:14PM DBG Note: this number might be smaller than expected due to commits with no additions
7:14PM INF scanned ~193 bytes (193 bytes) in 150ms
7:14PM WRN leaks found: 2
```

Por que isso é perigoso. Um segredo versionado continua no histórico do git mesmo depois de apagado do arquivo, e num repositório público é copiado por robôs em minutos. Com uma chave AWS válida um atacante lê buckets, sobe máquinas para minerar criptomoeda na conta da empresa ou apaga infraestrutura; é exatamente o incidente do cenário. A proteção correta é nunca escrever o segredo no código (variáveis de ambiente, secrets do GitHub, cofre) e barrar no pipeline qualquer commit que o contenha, para o vazamento não chegar a existir.

Achado extra: o próprio GitHub também barrou

Antes de o pipeline rodar, o `git push` do branch sabotado foi recusado pelo *push protection* do secret scanning do GitHub, ligado por padrão em repositórios públicos. O commit nem chegou ao servidor. Para a prova existir foi preciso liberar explicitamente os dois segredos pela interface do GitHub (motivo "usado em testes"), o que fica auditado no repositório. É defesa em profundidade: duas camadas independentes (GitHub e Gitleaks) detectaram o mesmo problema.

```
error: GH013: Repository rule violations found for refs/heads/sabotagem/segredo.
- GITHUB PUSH PROTECTION
  ——————————————————————————————————
    Resolve the following violations before pushing again
    - Push cannot contain secrets
     (?) Learn how to resolve a blocked push
      —— Amazon AWS Access Key ID ——————————————————
       locations:
         - commit: 19285e534ca2793039bb7b48729873755ef30832
           path: config/aws.env:3
       (?) To push, remove secret from commit(s) or follow this URL to allow the secret.
      —— Amazon AWS Secret Access Key ——————————————
       locations:
         - commit: 19285e534ca2793039bb7b48729873755ef30832
           path: config/aws.env:4
       (?) To push, remove secret from commit(s) or follow this URL to allow the secret.
To https://github.com/jonzaka/cp05-devsecops-pipeline.git
 ! [remote rejected] sabotagem/segredo -> sabotagem/segredo (push declined due to repository rule violations)
error: failed to push some refs to 'https://github.com/jonzaka/cp05-devsecops-pipeline.git'
```

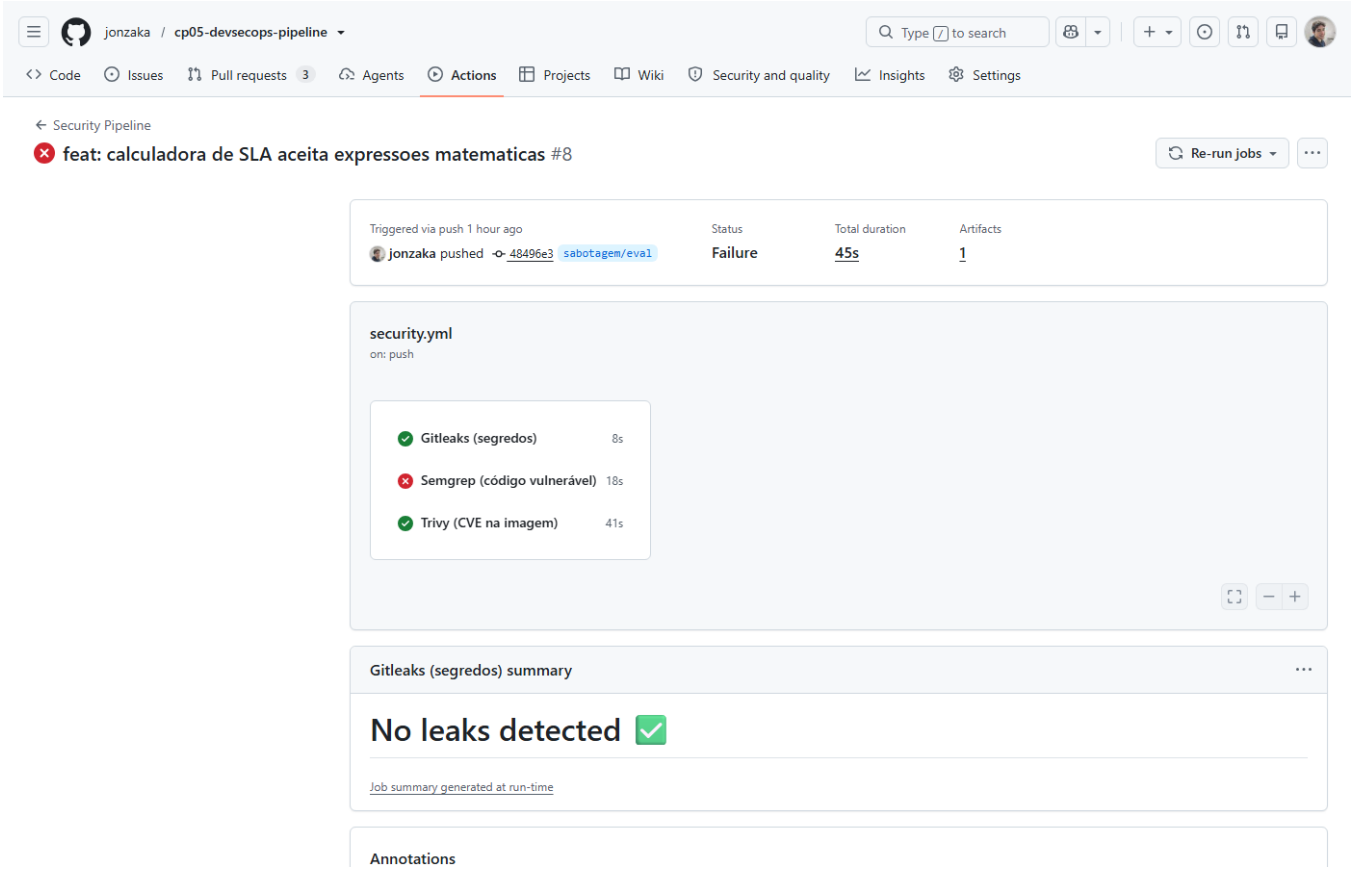
Saída do git push recusado pelo push protection do GitHub (links de liberação omitidos).

4. Prova 2 — código vulnerável apontado pelo Semgrep

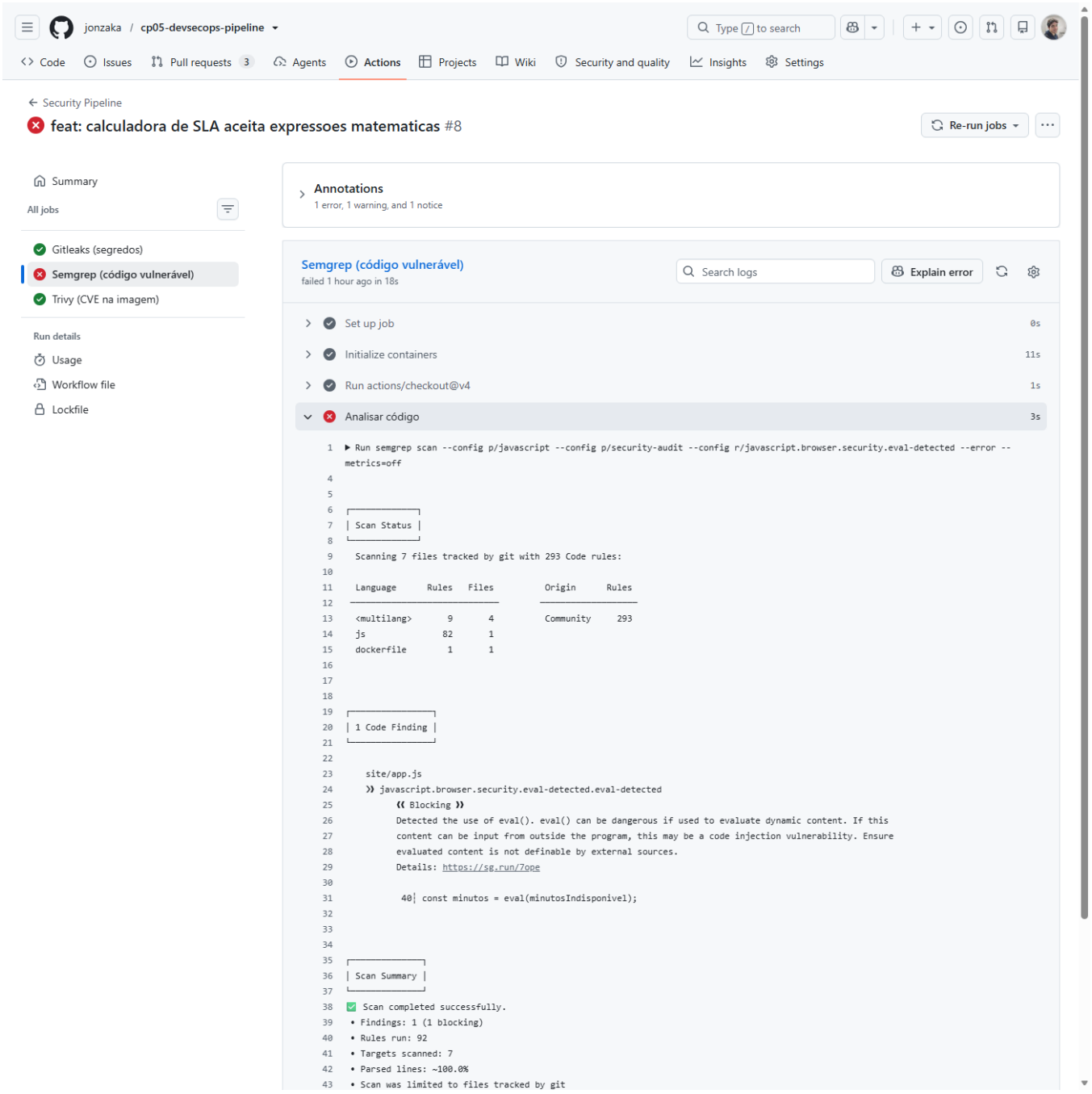
Sabotagem. No branch `sabotagem/eval` a calculadora de SLA do portal foi "melhorada" para aceitar expressões como `2*60` no campo de minutos. A implementação trocou `Number(minutosIndisponivel)` por `eval(minutosIndisponivel)`, e o campo do formulário passou de `type="number"` para `type="text"`. Diff de três linhas, sabotagem completa.

```
-  const minutos = Number(minutosIndisponivel);
+  // "Melhoria": aceita expressões como "2*60" no campo de minutos.
+  const minutos = eval(minutosIndisponivel);
```

Resultado. O job `semgrep` falhou com um achado bloqueante da regra `javascript.browser.security.eval-detected` na linha 40 de `site/app.js`; Gitleaks e Trivy passaram.



Run #8 do branch sabotagem/eval: só o Semgrep reprovou.



Job Semgrep: o passo "Analisar código" falhou.

Trecho do log do job

```
    site/app.js
    >> javascript.browser.security.eval-detected.eval-detected
          << Blocking >>
          Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
          content can be input from outside the program, this may be a code injection vulnerability. Ensure
          evaluated content is not definable by external sources.
          Details: https://sg.run/7ope
           40| const minutos = eval(minutosIndisponivel);
+--------------+
| Scan Summary |
+--------------+
[ok] Scan completed successfully.
 - Findings: 1 (1 blocking)
 - Rules run: 92
 - Targets scanned: 7
 - Parsed lines: ~100.0%
 - Scan was limited to files tracked by git
 - For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 92 rules on 7 files: 1 finding.
```

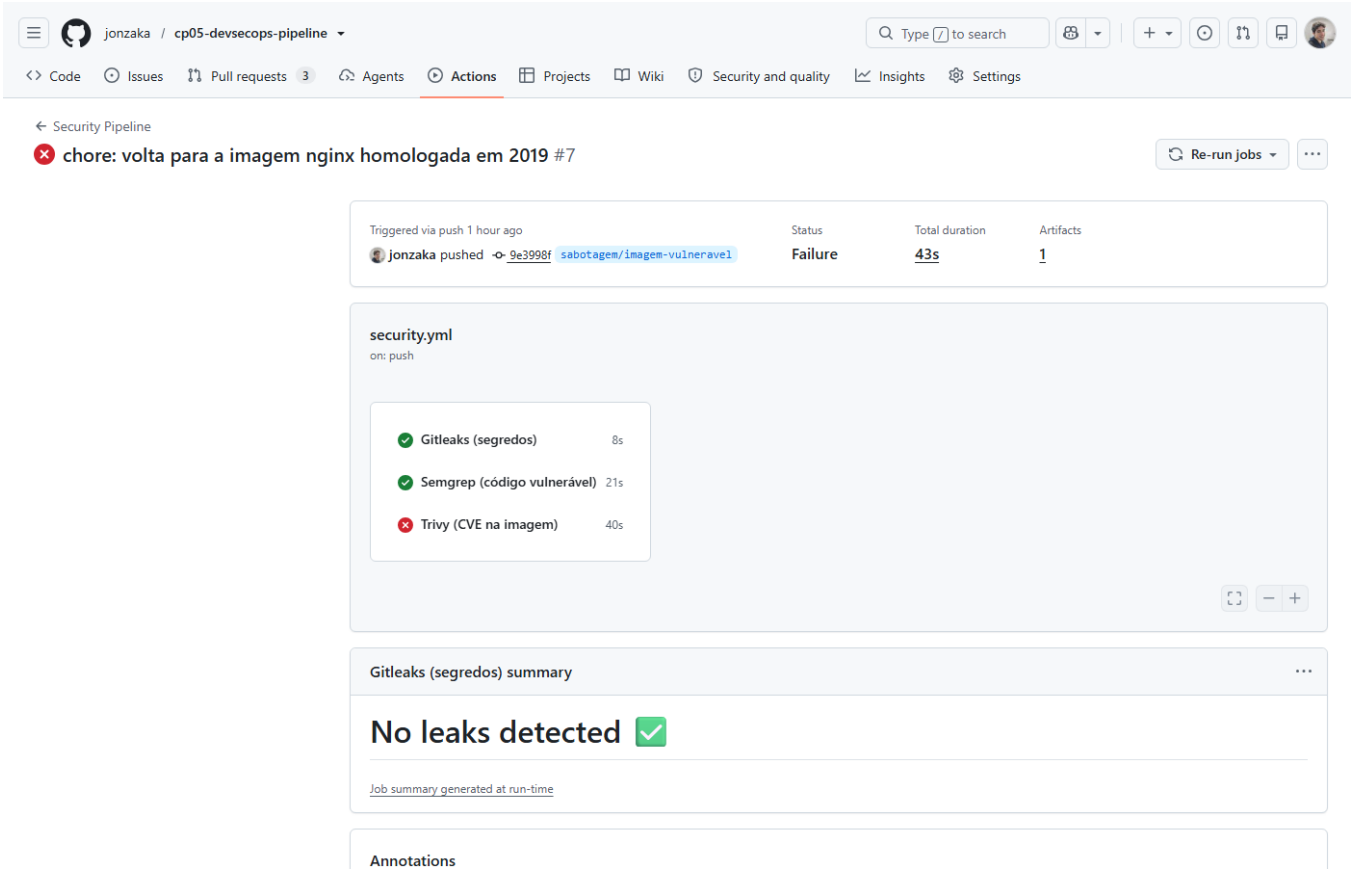
Por que isso é perigoso. `eval()` executa como código JavaScript qualquer texto que receber. Quando esse texto vem do usuário, quem digita no formulário passa a programar a página: pode ler cookies e tokens de sessão, enviar dados para outro domínio ou alterar o que outros usuários veem. É a injeção de código (CWE-95) que a OWASP classifica entre os riscos mais críticos. A correção é a versão original: converter com `Number()` e validar o intervalo, sem nunca interpretar entrada como código.

> **Decisão registrada durante a construção.** Na primeira execução o Semgrep rodou 91 regras e não acusou o `eval`. A regra `detect-eval-with-expression` dos packs é de análise de fluxo (taint) e só dispara quando a origem do dado é uma fonte conhecida, como `location.href`; um parâmetro de função não é. Por isso a regra `eval-detected`, que marca qualquer `eval(...)` com argumento não literal, foi adicionada explicitamente ao job. Lição: o pack padrão não é garantia; é preciso provar que a regra pega o caso.

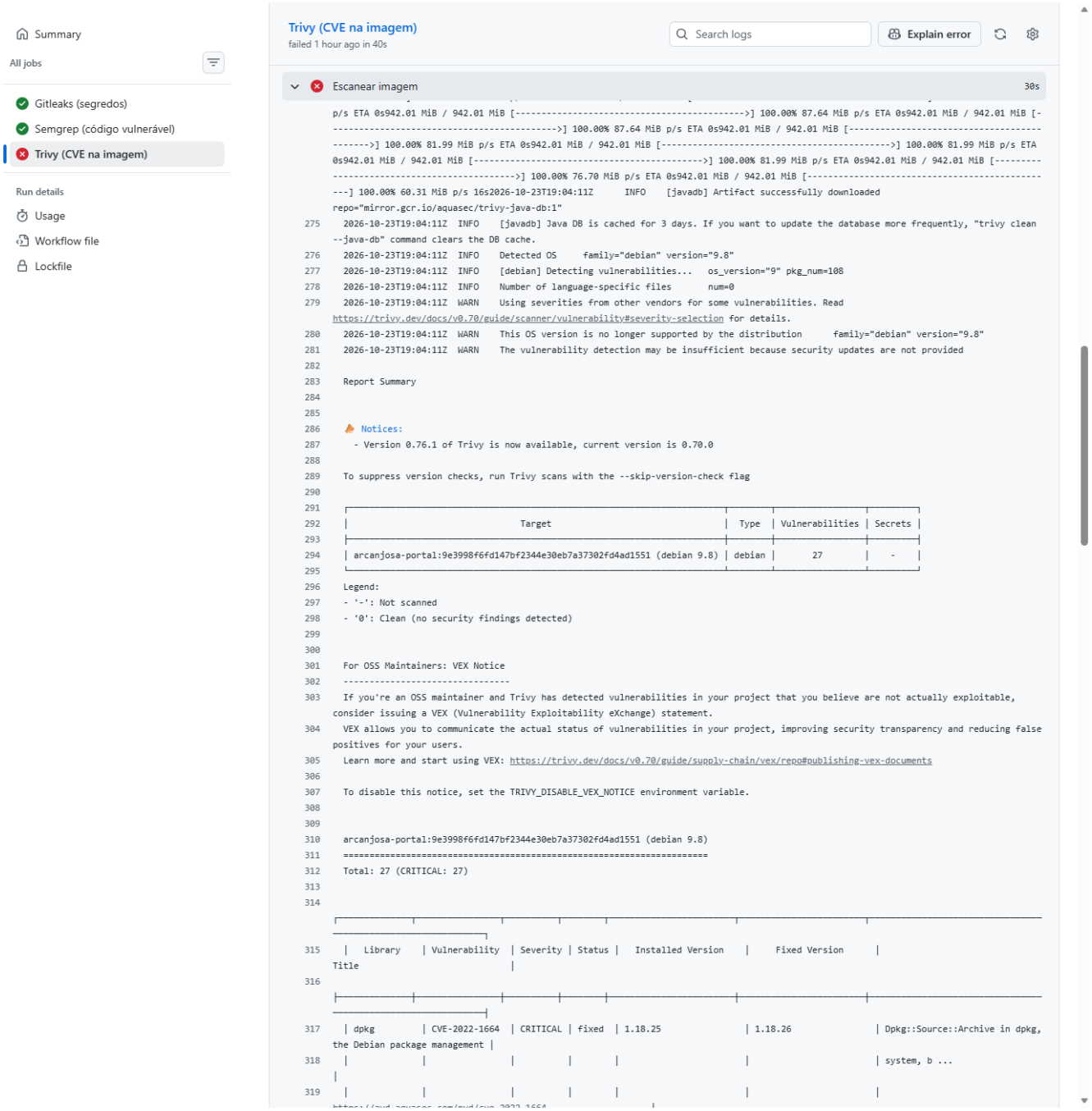
5. Prova 3 — imagem com CVE crítico barrada pelo Trivy

Sabotagem. No branch `sabotagem/imagem-vulneravel` o `Dockerfile` voltou a usar `nginx:1.14`, imagem de 2019 baseada em Debian 9, com a justificativa fictícia de ser "a versão homologada".

Resultado. O job `trivy` construiu a imagem, escaneou e encontrou 27 vulnerabilidades CRITICAL com correção disponível, no sistema operacional da imagem (dpkg, glibc, OpenSSL, entre outros). Com `exit-code: 1` e `severity: CRITICAL`, o job falhou; Gitleaks e Semgrep passaram.



Run #7 do branch sabotagem/imagem-vulneravel: só o Trivy reprovou.



Job Trivy: o passo "Escanear imagem" falhou.

Trecho do log do job

```
Total: 27 (CRITICAL: 27)
```

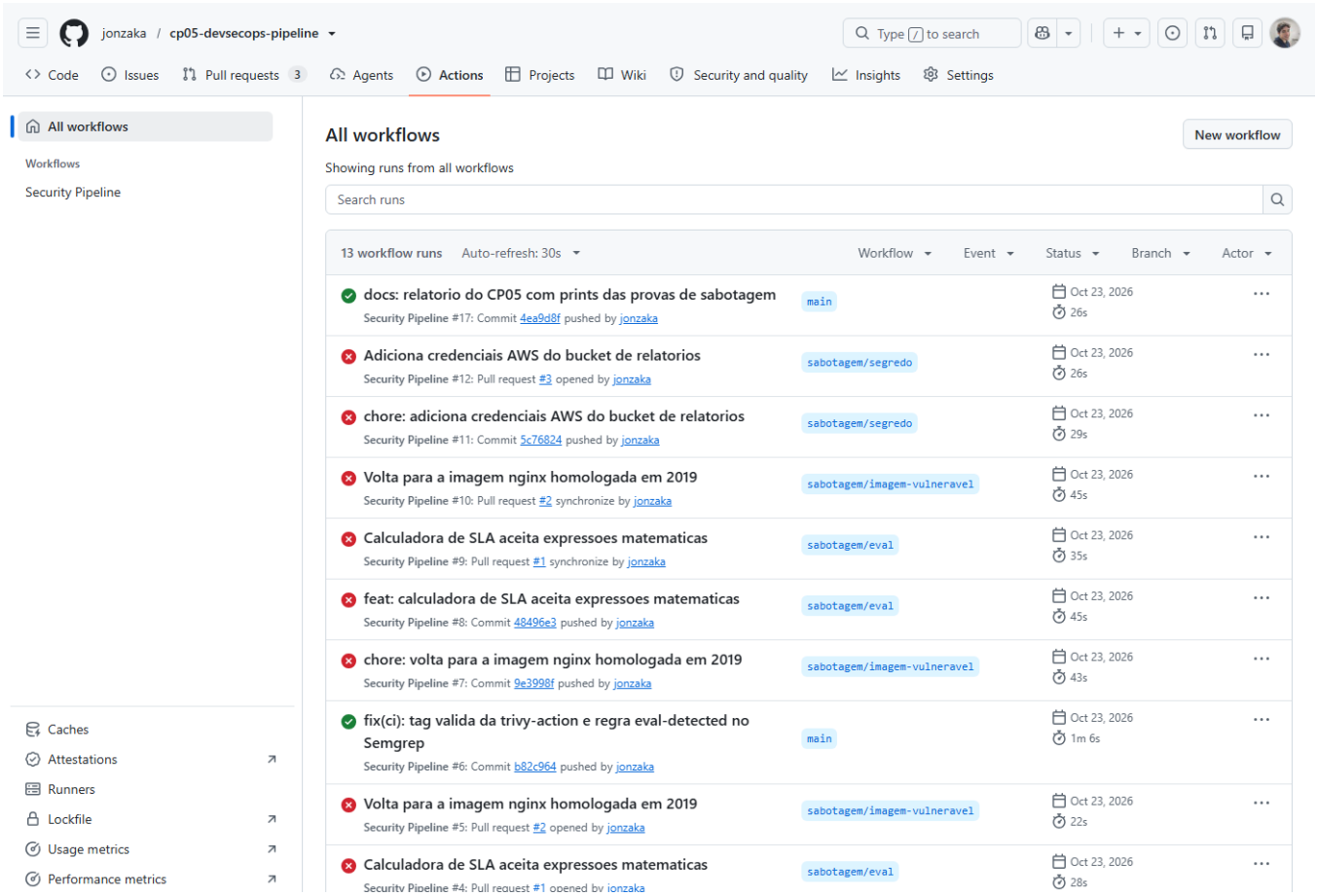
Biblioteca	CVE	Instalada	Corrigida em	Título (início)
dpkg	CVE-2022-1664	1.18.25	1.18.26	Dpkg::Source::Archive in dpkg, the Debian package management
libbsd0	CVE-2019-20367	0.8.3-1	0.8.3-1+deb9u1	nlist.c in libbsd before 0.10.0 has an out-of-bounds read
libexpat1	CVE-2022-22822	2.2.0-2+deb9u1	2.2.0-2+deb9u4	expat: Integer overflow in addBinding in xmlparse.c
libexpat1	CVE-2022-22823			expat: Expat: Denial of Service vulnerability in XML parsing
libexpat1	CVE-2022-22824			expat: Expat: Denial of service and memory issues due to

Biblioteca	CVE	Instalada	Corrigida em	Título (início)
libexpat1	CVE-2022-23852		2.2.0-2+deb9u5	expat: Integer overflow in function XML_GetBuffer
libexpat1	CVE-2022-25235			expat: Malformed 2- and 3-byte UTF-8 sequences can lead to
libexpat1	CVE-2022-25236			expat: Namespace-separator characters in "xmlns[:prefix]"
libexpat1	CVE-2022-25315			expat: Integer overflow in storeRawNames()
libfreetype6	CVE-2020-15999	2.6.3-3.2	2.6.3-3.2+deb9u2	freetype: Heap-based buffer overflow due to integer
liblz4-1	CVE-2021-3520	0.0~r131-2+b1	0.0~r131-2+deb9u1	lz4: memory corruption due to an integer overflow bug caused
libwebp6	CVE-2018-25009	0.5.2-1	0.5.2-1+deb9u1	libwebp: out-of-bounds read in WebPMuxCreateInternal
libwebp6	CVE-2018-25010			libwebp: out-of-bounds read in ApplyFilter()
libwebp6	CVE-2018-25011			libwebp: heap-based buffer overflow in PutLE16()

Primeiras 14 linhas da tabela do Trivy no log do job; são 27 CVEs no total, todos CRITICAL e com correção disponível (status "fixed").

Por que isso é perigoso. A imagem Docker carrega um sistema operacional inteiro, e o código da aplicação pode estar impecável enquanto a biblioteca de TLS ou o gerenciador de pacotes da base tem exploração pública. Debian 9 não recebe mais correções, então as 27 falhas ficam para sempre. Um atacante usa uma delas para executar código no container e, a partir dali, alcançar a rede interna. A regra do pipeline é objetiva: CVE crítico com correção disponível bloqueia, porque a correção existe e basta atualizar a base.

6. Visão geral e resumo



Aba Actions do repositório: a main verde e todos os branches de sabotagem vermelhos, cada um pela sua barreira.

Branch	Sabotagem	Gitleaks	Semgrep	Trivy	PR
`main`	nenhuma	**passou**	**passou**	**passou**	—
`sabotagem/segredo`	chave AWS falsa em `config/aws.env`	**FALHOU**	**passou**	**passou**	#3
`sabotagem/eval`	`eval()` sobre entrada do usuário	**passou**	**FALHOU**	**passou**	#1
`sabotagem/imagem-vulneravel`	`FROM nginx:1.14`	**passou**	**passou**	**FALHOU**	#2

Cada sabotagem acionou exatamente a barreira que deveria, e nenhuma outra: as ferramentas são independentes e cobrem camadas diferentes (histórico do git, código-fonte, imagem de execução). Os três pull requests ficam abertos de propósito, como registro das provas, e não devem ser mesclados.

Links

Repositório: https://github.com/jonzaka/cp05-devsecops-pipeline

Workflow: https://github.com/jonzaka/cp05-devsecops-pipeline/blob/main/.github/workflows/security.yml

Actions: https://github.com/jonzaka/cp05-devsecops-pipeline/actions